Module title: Mobile application system and design

Date: On 16th March 2026

Flutter Lab 4: Consuming APIs in flutter

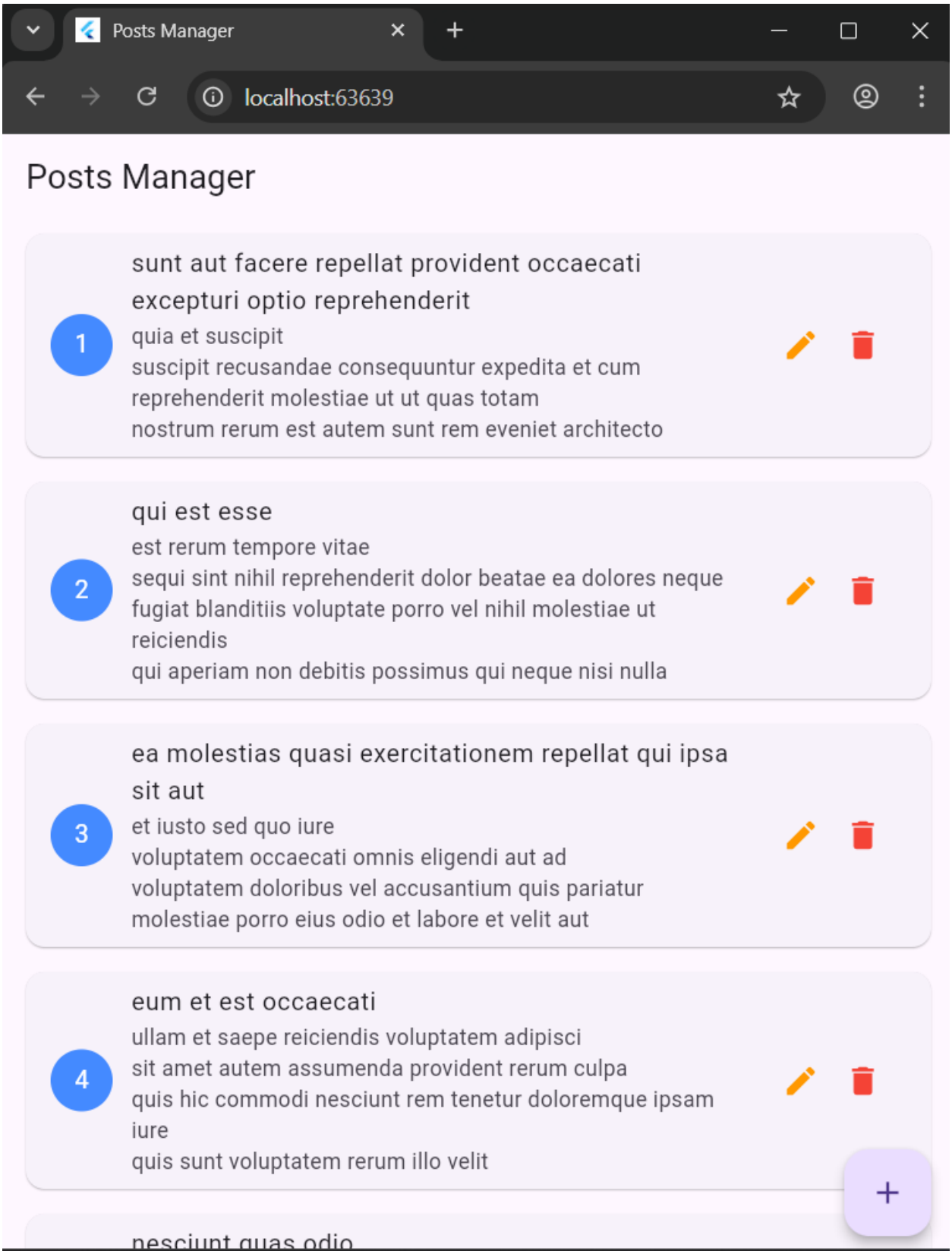
POSTS MANAGER

The **Posts Manager** Flutter app allows users to **view, create, edit, and delete posts** using a REST API. Posts have an **ID, title, and body**.

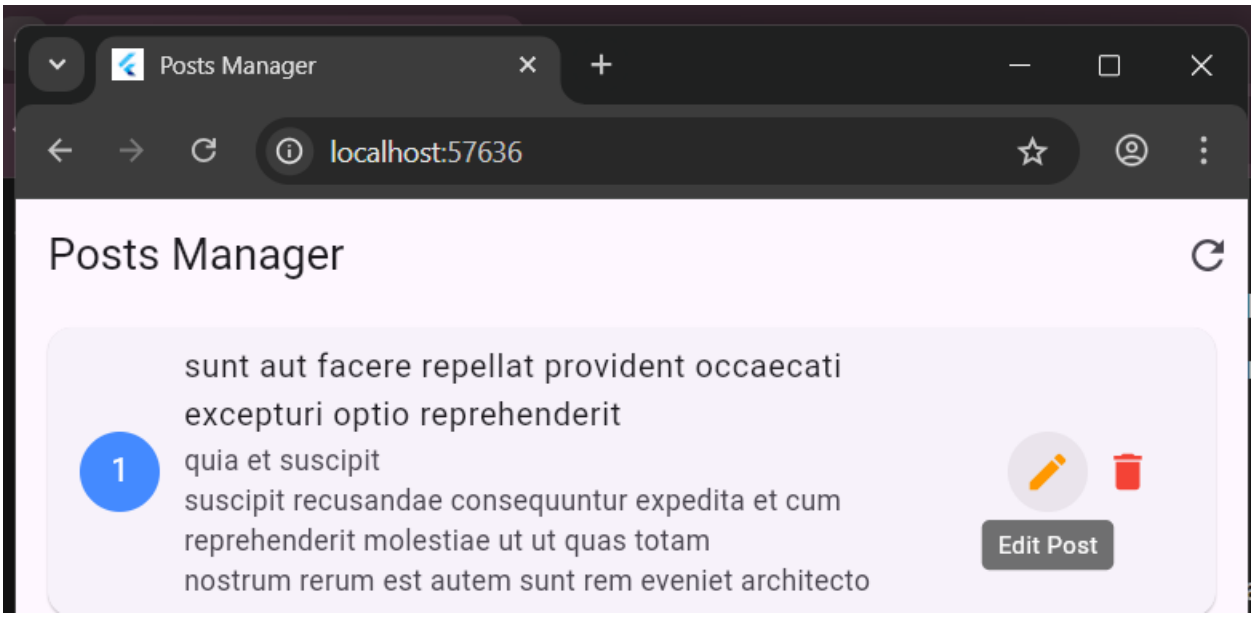
- **ID**: Unique identifier for the post
- **Title**: Title of the post
- **Body**: Content of the post

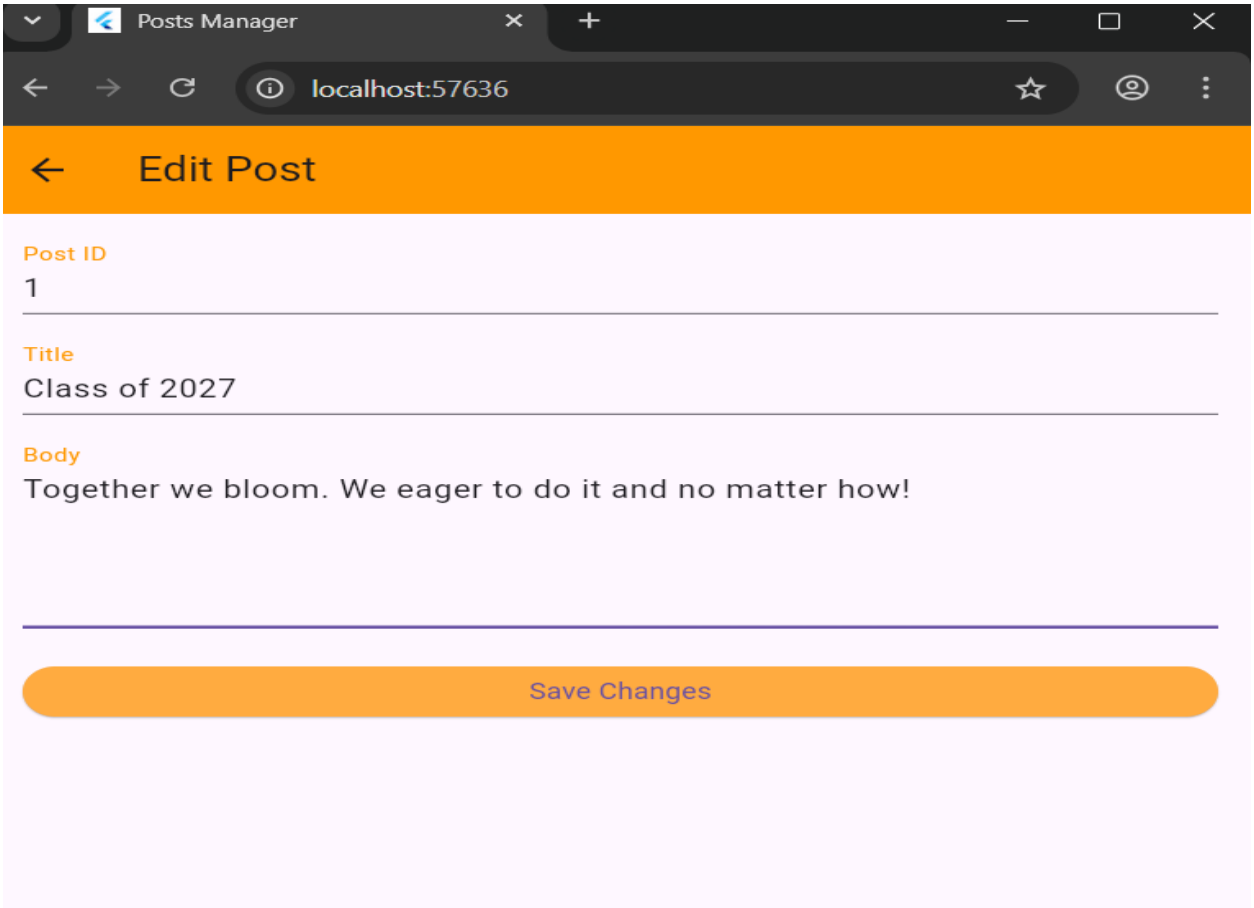
App Workflow

When launched, the app fetches posts from the API and displays them with ID badges, titles, and body content. Users can **create a post** via the "+" button, **edit a post** by clicking the edit icon (ID cannot change), or **delete a post** with a confirmation dialog. All actions update the list instantly. Tooltips on icons and blue ID badges provide a professional, user-friendly interface.

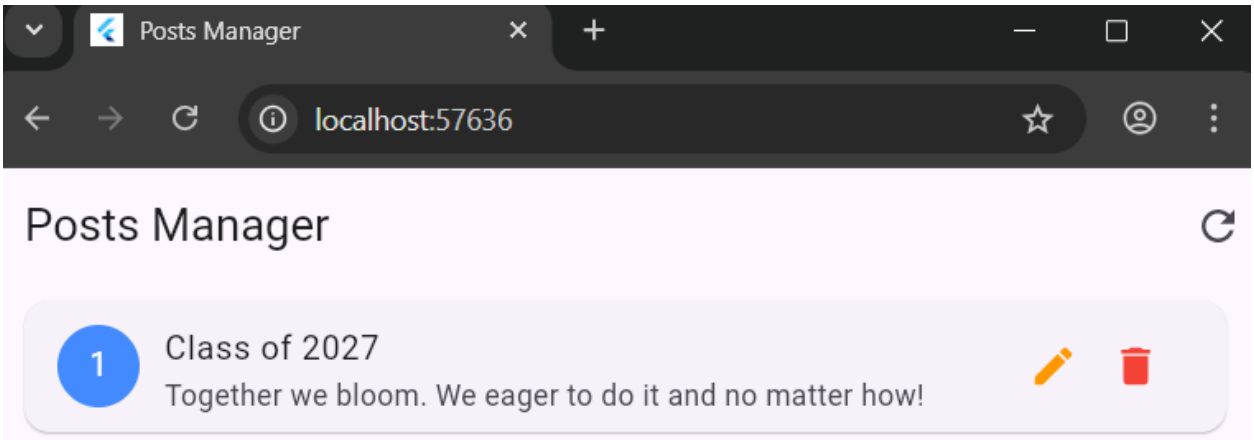

Posts Manager
localhost:63639
Posts Manager
1
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
quia et suscipit
suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto
2
qui est esse
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis
qui aperiam non debitis possimus qui neque nisi nulla
3
ea molestias quasi exercitationem repellat qui ipsa sit aut
et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad
voluptatem doloribus vel accusantium quis pariatur
molestiae porro eius odio et labore et velit aut
4
eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum culpa
quis hic commodi nesciunt rem tenetur doloremque ipsam iure
quis sunt voluptatem rerum illo velit
+
nesciunt quas odio

1. Edit post

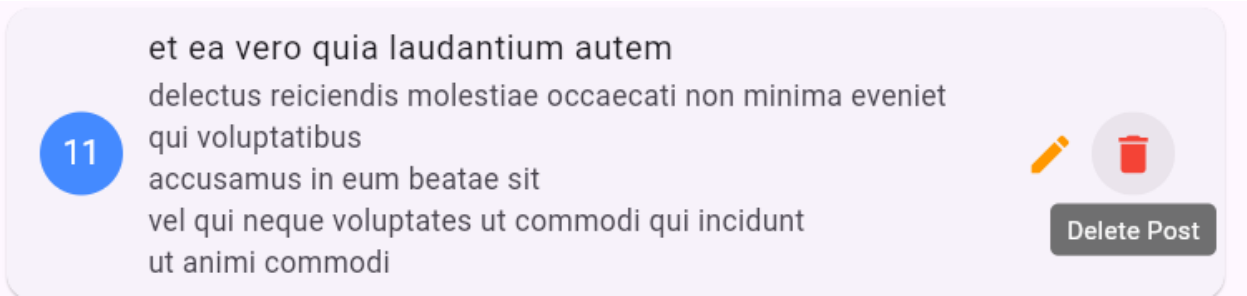

Posts Manager
localhost:57636
Posts Manager
1
sunt aut facere repellat provident occaecati
excepturi optio reprehenderit
quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto
Edit Post


Posts Manager
localhost:57636
Edit Post
Post ID
1
Title
Class of 2027
Body
Together we bloom. We eager to do it and no matter how!
Save Changes

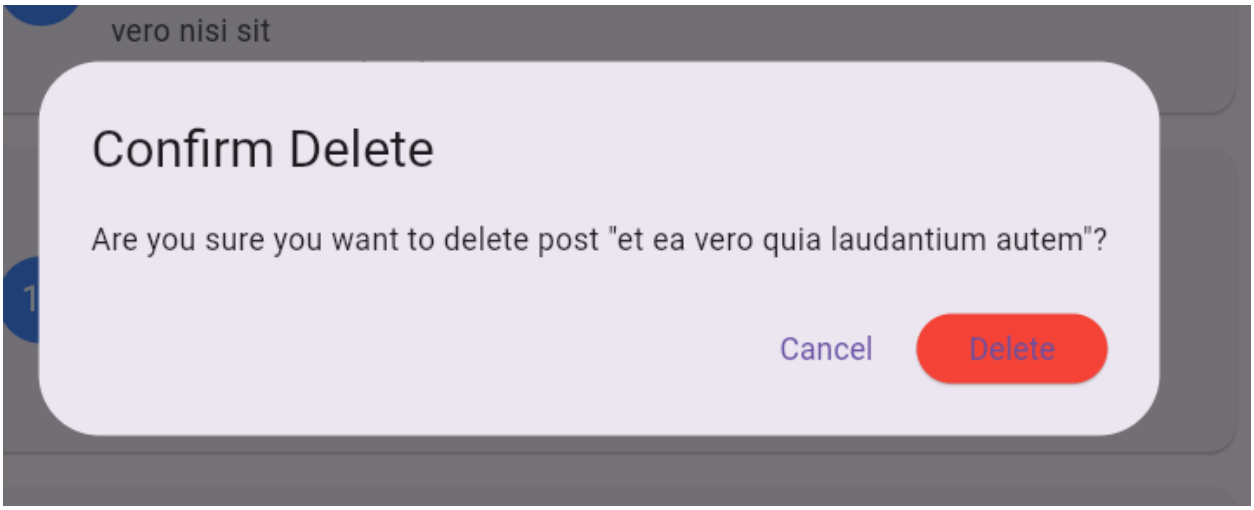
Edit Post: Update a post's title or body; ID remains the same; changes appear instantly in the list after saving changes.



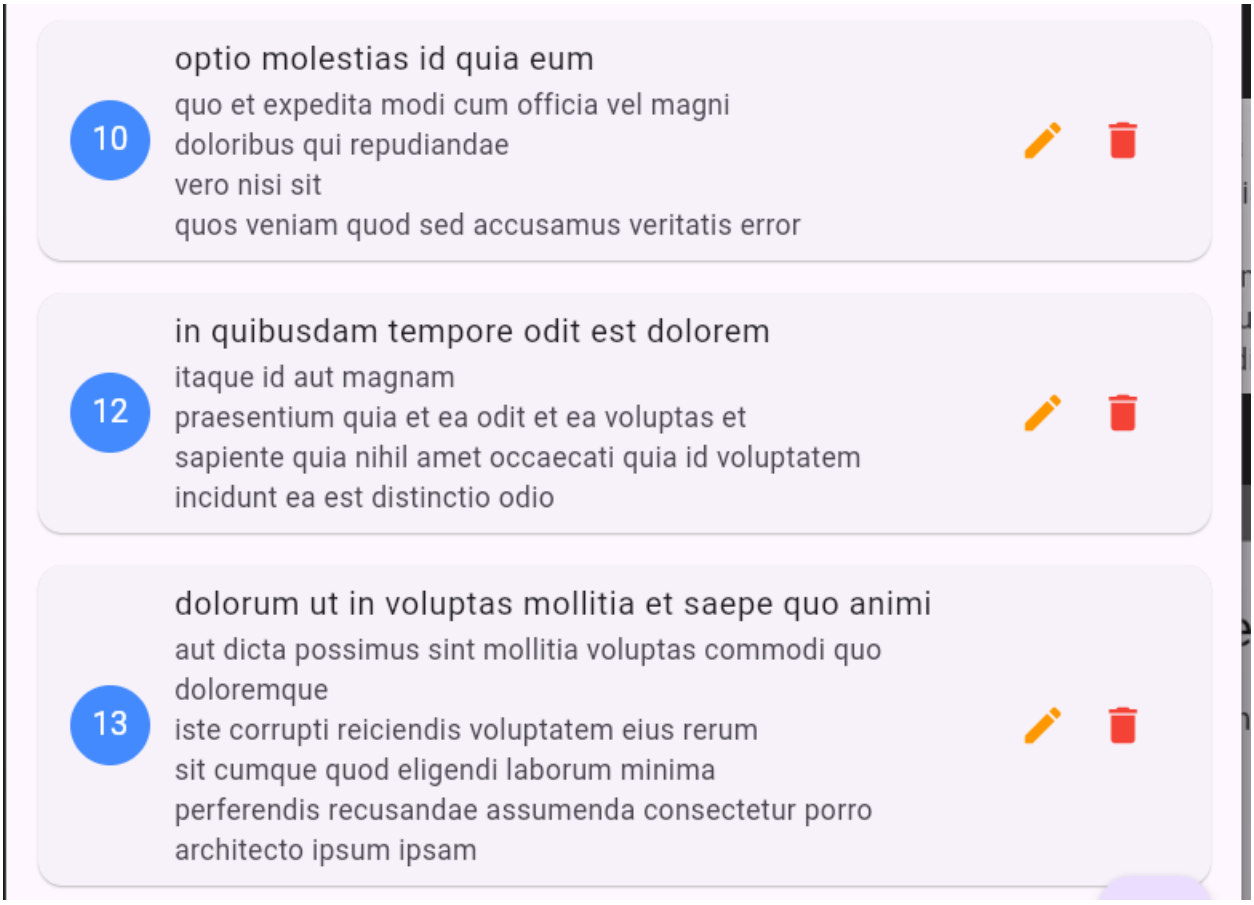
2. Delete post



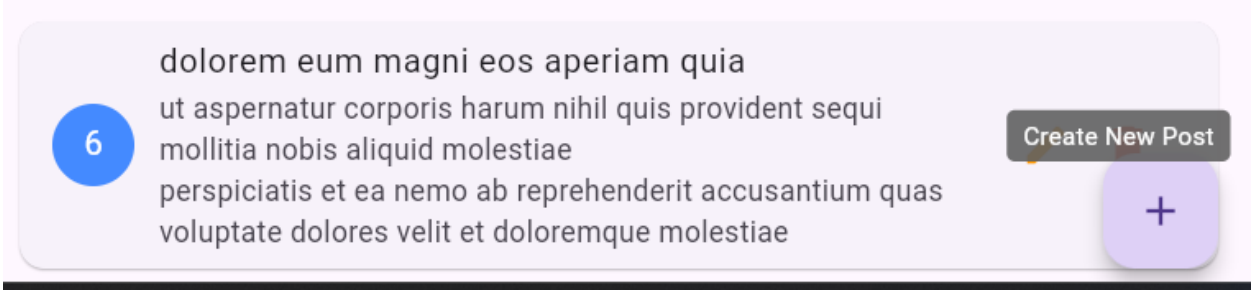
Let me delete post 11



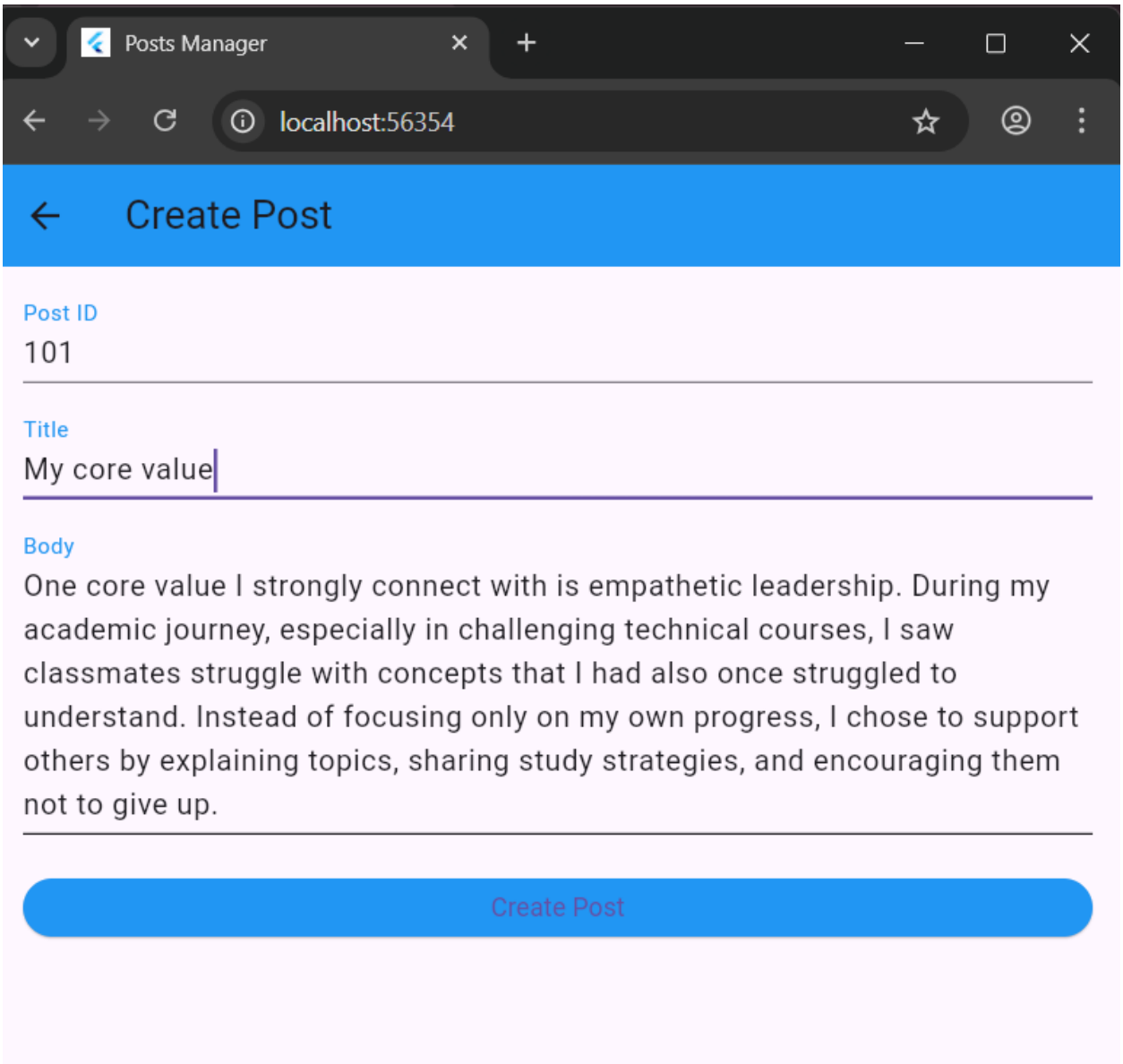
If I click delete, the post 11 will disappear from the list.

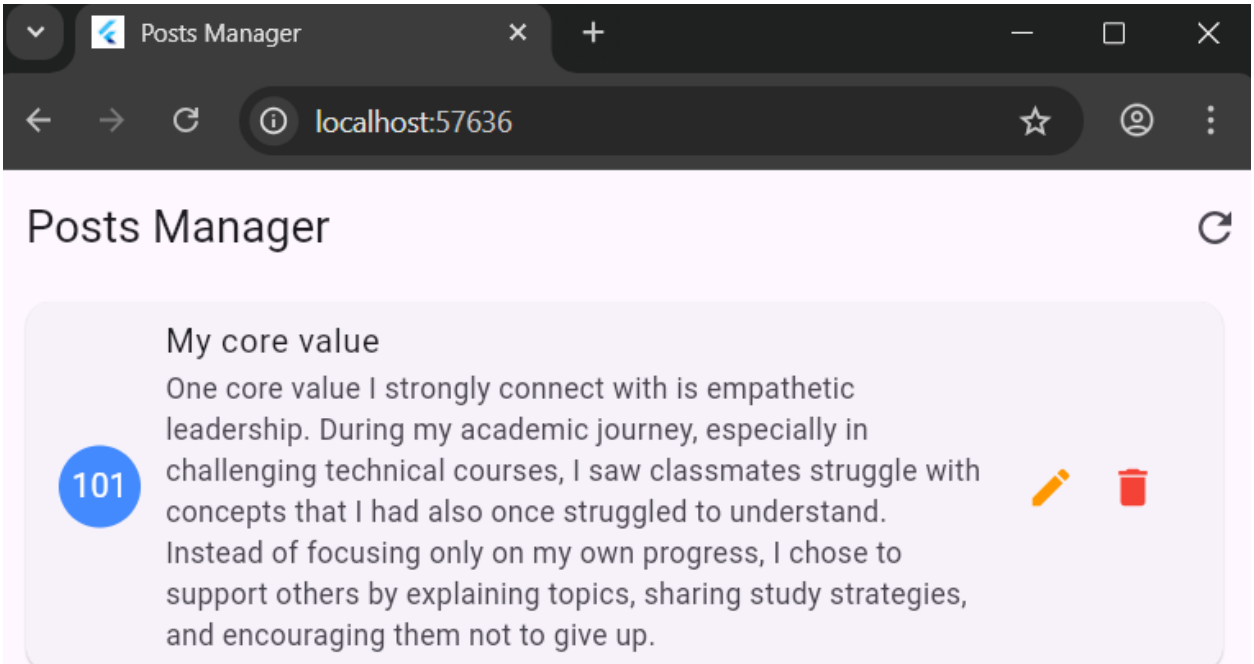


3. Create a post



Create Post: Add a new post with ID (unique), title, and body; appears instantly in the list.

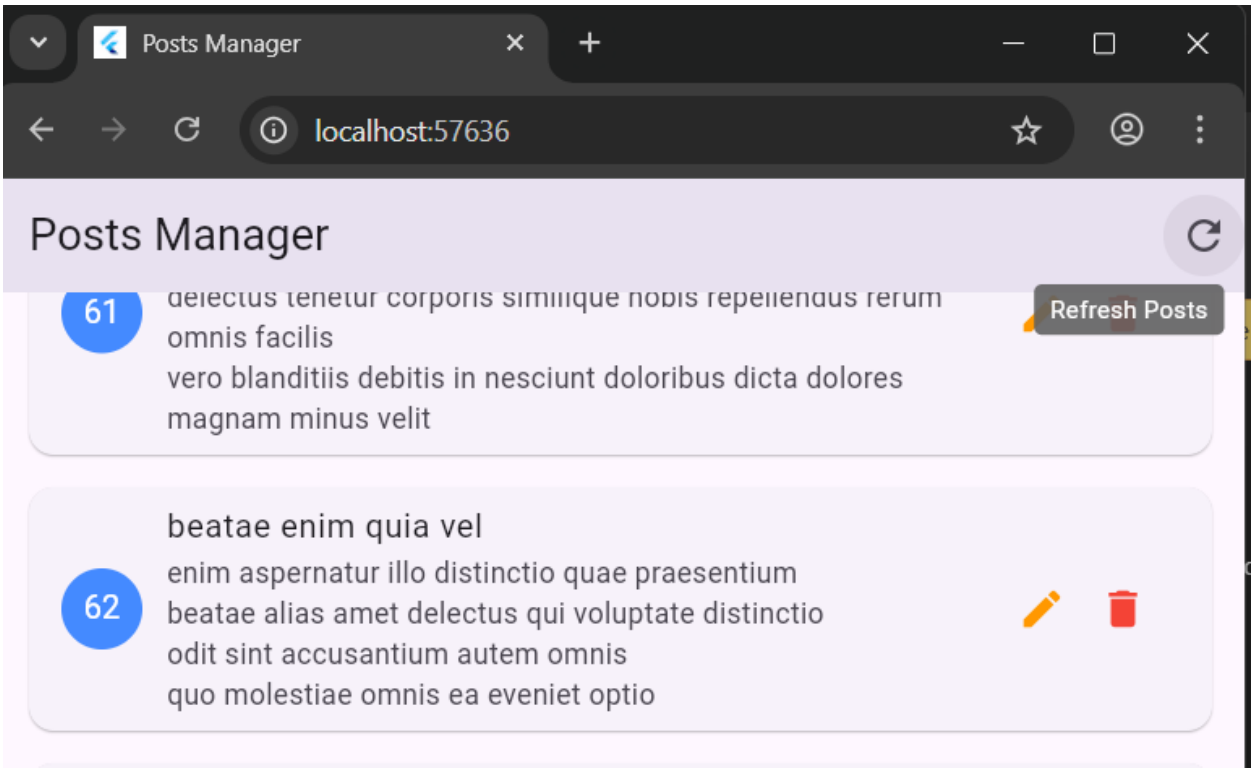

Posts Manager
localhost:56354
Create Post
Post ID
101
Title
My core value
Body
One core value I strongly connect with is empathetic leadership. During my academic journey, especially in challenging technical courses, I saw classmates struggle with concepts that I had also once struggled to understand. Instead of focusing only on my own progress, I chose to support others by explaining topics, sharing study strategies, and encouraging them not to give up.
Create Post



The post is already on the list.

4. Refresh

Refresh: Reload the posts list from the API to see the latest updates.



Question 1. Dependencies I used and why.

> http: It performs REST calls to JSONPlaceholder (GET / POST / PUT / DELETE.

Why?

- Because it is the standard, lightweight Dart package for making HTTP requests and handling JSON responses.
  - GET request retrieves all posts from the API.
  - POST request creates a new post.
  - PUT request updates an existing post.
  - DELETE request removes a post from the server.

> flutter / cupertino_icons

- Flutter: A core framework required for every Flutter App.
- cupertino_icons: provides iOS-style icons (used by default in many templates).

Question 2. How to handle API exceptions (network errors, invalid data)

> Network errors can be handled using try-catch blocks to detect when the internet connection fails.

> Invalid responses can be checked using HTTP status code. For instance, if the status code is not 200, an error message can be shown.

> User feedback can be provided by displaying messages such as "Failed to load data!" or showing a loading indicator while waiting for the API response.

Question 3. FutureWidget used and UI dependence on it.

> Widget used: FutureBuilder <T> : It is used to load the post list from the network in postListPage.

* Why FutureBuilder? Because it rebuilds UI automatically based on the asynchronous state to the API call. (it supports three main UI states: Loading, Error, data ready).

* UI parts depending on the future: Loading UI (while waiting for the future to comple), error UI ("Try again"), data UI (ListView of posts)